

MAKALAH INDIVIDUAL / NILAI TAMBAHAN

PANDUAN PRAKTIS DAN ANALISIS LANGKAH PENGERJAAN SERTIFIKASI NEO4J CERTIFIED PROFESSIONAL SECARA MANDIRI

Disusun Oleh:

Nama: Gading Nasution

Mata Kuliah: Data Science / Basis Data Lanjut

Program Studi: Informatika

DAFTAR ISI

1. **BAB I: PENDAHULUAN**
 - 1.1 Latar Belakang
 - 1.2 Rumusan Masalah
 - 1.3 Tujuan Penulisan
2. **BAB II: LANDASAN TEORI GRAPH DATABASE & NEO4J**
 - 2.1 Konsep Dasar Graph Database vs Relational Database (RDBMS)
 - 2.2 Property Graph Model
 - 2.3 Analisis Performa: *Index-Free Adjacency* vs *Relational Join* $O(n)$
3. **BAB III: LINGKUNGAN KERJA DAN PERSIAPAN SERTIFIKASI**
 - 3.1 Persiapan Akun & Platform Neo4j GraphAcademy
 - 3.2 Setup Neo4j Sandbox / DBMS Local
 - 3.3 Konfigurasi Neo4j Driver & Protokol Koneksi Aman
4. **BAB IV: BEDAH MATERI DAN KISI-KISI UJIAN (6 PILAR UTAMA)**
 - 4.1 Pilar 1: Fundamental Graph Model & Relationship Uniqueness
 - 4.2 Pilar 2: Sintaks & Operasi Cypher Query Language
 - 4.3 Pilar 3: Schema Constraints & Indexing (Node Key, Vector, Text Index)
 - 4.4 Pilar 4: Neo4j Drivers & Security Protocols
 - 4.5 Pilar 5: Query Profiling & Performance Tuning (Analysis of Red Flags)
 - 4.6 Pilar 6: Eksplorasi Query Sandbox & Agregasi Data Hands-on
5. **BAB V: STRATEGI DAN LANGKAH PENGERJAAN SERTIFIKASI SECARA MANDIRI**
 - 5.1 Estimasi & Manajemen Waktu Ujian (80 Soal)

- 5.2 Teknik Membaca Soal & Mengurai Pola Cypher Pattern Matching
 - 5.3 Langkah Eksekusi Validasi Query via Neo4j Browser / Query Editor
6. **BAB VI: PENUTUP**
- 6.1 Kesimpulan
 - 6.2 Saran
-

BAB I: PENDAHULUAN

1.1 Latar Belakang

Perkembangan volume dan kompleksitas data pada era *Data Science* saat ini menuntut arsitektur penyimpanan data yang mampu menangani hubungan terhubung (*highly interconnected data*) secara efisien. Database relasional tradisional (RDBMS) yang mengandalkan tabel, kunci asing (*foreign key*), dan operasi *JOIN* seringkali mengalami penurunan performa secara drastis saat memproses penelusuran hubungan bernivel banyak (*deep traversal*) pada skala data yang besar.

Neo4j hadir sebagai platform *Graph Database* terkemuka yang menerapkan model *Property Graph*. Untuk mengukur dan membuktikan kompetensi teknis seorang profesional data science maupun *data engineer* dalam mengelola database graph, Neo4j menyelenggarakan program sertifikasi resmi **Neo4j Certified Professional**. Sertifikasi ini menguji pemahaman komprehensif mulai dari pemodelan graph, bahasa kueri *Cypher*, optimasi indeks, manajemen transaksi, hingga konektivitas *Driver* aplikasi.

Makalah ini disusun sebagai dokumentasi analitis dan panduan langkah-langkah praktis pengerjaan sertifikasi Neo4j secara mandiri. Makalah ini juga ditujukan sebagai pemenuhan komponen tugas / nilai tambahan pada mata kuliah Data Science.

1.2 Rumusan Masalah

1. Bagaimana perbedaan mendasar antara arsitektur Graph Database Neo4j dan Database Relasional dalam menangani relasi antar data?
2. Apa saja materi inti (6 pilar utama) yang diuji pada ujian sertifikasi Neo4j Certified Professional?
3. Bagaimana strategi dan langkah-langkah sistematis untuk menyelesaikan ujian sertifikasi Neo4j Certified Professional berjumlah 80 soal secara mandiri dengan hasil yang optimal?

1.3 Tujuan Penulisan

1. Memahami arsitektur pemodelan data *Property Graph* serta prinsip kerja *Index-Free Adjacency*.
 2. Membedah kisi-kisi dan materi teknis ujian sertifikasi Neo4j Certified Professional berdasarkan skenario kasus nyata.
 3. Menyediakan panduan langkah demi langkah (*step-by-step guide*) pengerjaan ujian sertifikasi secara mandiri bagi mahasiswa dan praktisi.
-

BAB II: LANDASAN TEORI GRAPH DATABASE & NEO4J

2.1 Konsep Dasar Graph Database vs Relational Database (RDBMS)

Graph Database adalah sistem manajemen basis data yang menempatkan **relasi antar data sama pentingnya dengan data itu sendiri**.

Perbandingan arsitektural utama antara RDBMS dan Graph Database dapat diringkas sebagai berikut:

- **Entity / Data Object:** Di RDBMS disimpan dalam bentuk *Baris (Row)* di dalam *Tabel*. Di Neo4j disimpan sebagai **Node**.
- **Kategori / Tipe Data:** Di RDBMS berupa *Nama Tabel*. Di Neo4j berupa **Label** (misal :Person, :Movie).
- **Hubungan / Relasi:** Di RDBMS menggunakan *Foreign Key* dan *Join Table*. Di Neo4j berupa **Relationship** eksplisit yang memiliki tipe, arah, dan properti.
- **Atribut:** Di RDBMS berupa *Kolom*. Di Neo4j berupa **Property** (key-value pair) yang melekat pada Node maupun Relationship.

2.2 Property Graph Model

Model *Property Graph* pada Neo4j terdiri dari empat elemen inti: 1. **Node:** Entitas utama (obyek diskrit) dalam domain data. 2. **Label:** Pengelompokan tipe/kategori node. Satu node dapat memiliki **nol, satu, atau banyak label** sekaligus (misal :Person:Actor:Director). 3. **Relationship:** Hubungan terarah (*directed*) antar dua node. Sebuah relasi **selalu memiliki tepat satu tipe** (misal :ACTED_IN, :DIRECTED), tidak boleh berupa multi-label, serta memiliki node asal (*start node*) dan node tujuan (*end node*). 4. **Properties:** Pasangan nama dan nilai (*key-value*) yang dapat ditambahkan pada Node maupun Relationship untuk menyimpan detail atribut (misalnya {title: 'Toy Story', released: 1995}).

2.3 Analisis Performa: *Index-Free Adjacency* vs *Relational Join* $O(n)$

Salah satu pertanyaan fundamental pada ujian sertifikasi Neo4j adalah mengenai masalah performa RDBMS yang dikenal sebagai **The $O(n)$ Problem**.

- **RDBMS ($O(n)$ / $O(\log n)$ per join)**: Pada database relasional, untuk menelusuri hubungan antar tabel, sistem harus membaca atau mencocokkan nilai kunci menggunakan indeks tabel (*B-Tree index seek/scan*). Seiring bertambahnya jumlah data n , waktu eksekusi kueri *JOIN* akan meningkat secara linear atau logaritmik mengikuti ukuran total tabel.
 - **Neo4j ($O(1)$ per hop / Index-Free Adjacency)**: Neo4j mengimplementasikan **Index-Free Adjacency**, di mana setiap node menyimpan penunjuk memori fisik (*direct memory pointers*) secara langsung ke node-node tetangganya. Penelusuran relasi dari satu node ke node berikutnya berjalan dalam waktu konstan $O(1)$ tanpa perlu melakukan *index lookup*. Waktu eksekusi kueri hanya bergantung pada jumlah subgraph yang ditelusuri, bukan pada ukuran total keseluruhan database.
-

BAB III: LINGKUNGAN KERJA DAN PERSIAPAN SERTIFIKASI

3.1 Persiapan Akun & Platform Neo4j GraphAcademy

1. Buka peramban (*web browser*) dan akses portal resmi **Neo4j GraphAcademy** di <https://graphacademy.neo4j.com>.
2. Lakukan pendaftaran akun (*Sign Up*) atau *Log In* menggunakan SSO Google/GitHub.
3. Masuk ke menu **Certifications** dan pilih **Neo4j Certified Professional**.
4. Ujian ini bersifat *free-of-charge* (gratis), berdurasi **60 menit**, terdiri dari **80 soal pilihan ganda / multi-select**, dengan ambang batas kelulusan **80%**.

3.2 Setup Neo4j Sandbox / DBMS Local

Beberapa pertanyaan ujian memerlukan verifikasi kueri atau penghitungan jumlah data secara langsung (*hands-on query*). 1. Manfaatkan **Neo4j Sandbox** (<https://sandbox.neo4j.com>) dengan memilih dataset sampel (misalnya *Movie Database* atau *Northwind Recommendation*). 2. Alternatif lain, gunakan **Neo4j Desktop** atau container **Docker Neo4j**: `bash docker run -d --name neo4j-cert -p 7474:7474 -p 7687:7687 -e`

NEO4J_AUTH=neo4j/passwordTest neo4j:latest 3. Akses Neo4j Browser melalui peramban di <http://localhost:7474> untuk menjalankan kueri pengujian.

3.3 Konfigurasi Neo4j Driver & Protokol Koneksi Aman

Untuk menghubungkan aplikasi (seperti Python, Java, JavaScript, .NET, Go) ke server Neo4j, aplikasi menggunakan library **Driver Neo4j**. Urutan siklus hidup driver yang benar adalah:

Connect to server → Verify connection → Execute Cypher statement → Parse results → Close connection

Skema URI resmi untuk membuat koneksi aman (*secure connection*) pada Driver Neo4j: - **neo4j+s** / **bolt+s**: Koneksi terenkripsi dengan pengujian sertifikat CA penuh (*Full Certificate Check against System CA Store*). - **neo4j+ssc** / **bolt+ssc**: Koneksi terenkripsi untuk sertifikat *self-signed* (*Self-Signed Certificate, no CA check*).

BAB IV: BEDAH MATERI DAN KISI-KISI UJIAN (6 PILAR UTAMA)

4.1 Pilar 1: Fundamental Graph Model & Relationship Uniqueness

- **Relationship Uniqueness:** Di dalam satu klausa MATCH Cypher, sebuah relasi fisik di database **hanya boleh dilintasi tepat 1 kali** per pola matching (*pattern match*).
 - *Dampak:* Pada kueri MATCH (p:Person {name: 'Al Pacino'})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(costar) Cypher otomatis melarang costar bernilai Al Pacino itu sendiri karena relasi :ACTED_IN film tersebut tidak dapat dipakai dua kali dalam 1 klausa.
- **Karakteristik Label & Relasi:**
 - Node dapat memiliki 0, 1, atau banyak label.
 - Relasi hanya memiliki tepat 1 tipe relasi (*relationship type*).

4.2 Pilar 2: Sintaks & Operasi Cypher Query Language

- **Klausa Pemodelan & Filter:**
 - Filter rentang terbilang (inclusive): WHERE m.released >= 2000 AND m.released <= 2005.
 - Filter keanggotaan daftar list: WHERE p.born IN [1970, 1980, 1990].
- **Pengolahan Mutasi (MERGE vs CREATE):**
 - MERGE bertindak sebagai *get-or-create*. Jika node belum ada, MERGE akan membuat node baru.

- Klausa ON CREATE SET hanya dieksekusi saat penciptaan node baru, sedangkan ON MATCH SET hanya dieksekusi jika node sudah ada sebelumnya. Properti tanpa klausa kondisional akan selalu dieksekusi.
- **Importing Data (LOAD CSV):**
 - Mengimpor file CSV dengan header: LOAD CSV WITH HEADERS FROM 'file:///data.csv' AS row.
- **Subquery Batching:**
 - Membagi eksekusi transaksi besar per-batch (Neo4j 5+): CALL { ... } IN TRANSACTIONS OF 1000 ROWS.

4.3 Pilar 3: Schema Constraints & Indexing

- **Node Key Constraint:** Constraint paling kuat yang menjamin dua hal sekaligus:
 1. *Uniqueness*: Kombinasi properti bernilai unik.
 2. *Existence*: Semua properti tersebut wajib ada / tidak boleh bernilai null.
- **Type-Type Indeks Resmi di Neo4j:**
 - RANGE: Indeks standar untuk perbandingan kesamaan dan rentang nilai (angka, string, tanggal).
 - TEXT: Khusus pencarian string (misal STARTS WITH, ENDS WITH, CONTAINS).
 - VECTOR: Indeks pencarian vektor/embedding untuk AI/LLM (CREATE VECTOR INDEX).
 - FULLTEXT: Pencarian teks penuh Lucene (CALL db.index.fulltext.queryNodes(...)).
 - POINT & LOOKUP.

4.4 Pilar 4: Neo4j Drivers & Security Protocols

- Driver Neo4j berfungsi menyediakan *connection pooling* dan eksekusi transaksi terkelola.
- Protokol URI aman yang sah diuji: neo4j+s, bolt+s, dan neo4j+ssc.

4.5 Pilar 5: Query Profiling & Performance Tuning (Analysis of Red Flags)

Dalam mengevaluasi performa kueri Cypher menggunakan perintah EXPLAIN atau PROFILE, indikator bahaya (*Red Flags*) yang harus diidentifikasi meliputi: 1. **AllNodesScan**: Terjadi *full table scan* terhadap seluruh node di database akibat kurangnya label atau indeks. 2.

NodeByLabelScan dengan DB Hits tinggi: Menandakan pencarian label yang tidak spesifik tanpa dukungan indeks properti. 3. **Penempatan Eager Operators**: Eager operator (seperti Eager pada pengubahan data) menahan seluruh akumulasi hasil di memori sebelum meneruskannya ke tahapan

kueri berikutnya, yang memicu lonjakan penggunaan RAM dan potensi *Out of Memory*.

4.6 Pilar 6: Eksplorasi Query Sandbox & Agregasi Data Hands-on

Beberapa soal meminta penghitungan data secara presisi pada dataset sandbox yang disediakan: - **Menghitung Total Node:** cypher MATCH (u:User) RETURN count(u) - **Mencari Entitas dengan Dua Peran Sekaligus (Aktor & Sutradara):** cypher MATCH (p:Person)-[:ACTED_IN]->(m:Movie), (p)-[:DIRECTED]->(m) RETURN p.name, count(m) AS moviesCount ORDER BY moviesCount DESC LIMIT 1

BAB V: STRATEGI DAN LANGKAH Pengerjaan Sertifikasi Secara Mandiri

5.1 Estimasi & Manajemen Waktu Ujian (80 Soal)

- **Total Waktu:** 60 Menit (3.600 Detik).
- **Rata-rata Waktu per Soal:** ≈ 45 detik/soal.
- **Strategi Alokasi Waktu:**
 1. *Pass Pertama (Menit 0 - 25):* Jawab soal-soal konseptual (teori graph, protokol driver, definisi Cypher) yang dapat diselesaikan dalam 10-20 detik.
 2. *Pass Kedua (Menit 25 - 50):* Kerjakan soal logika Cypher pattern matching, evaluasi constraint, dan analisis query plan.
 3. *Pass Ketiga (Menit 50 - 60):* Gunakan Neo4j Browser/Sandbox untuk memverifikasi soal yang membutuhkan jawaban eksak angka/nama node.

5.2 Teknik Membaca Soal & Mengurai Pola Cypher Pattern Matching

1. **Perhatikan Jenis Pilihan:** Bedakan antara *Single Choice* (radio button) dan *Multi Select* (checkbox). Jika multi-select, baca seluruh opsi dengan teliti karena biasanya terdapat 2 atau 3 jawaban benar.
2. **Cermati Kata Kunci:**
 - Jika ada kata "*unique AND exist*" → Jawaban pasti **Node Key Constraint**.
 - Jika ada kata "*faster traversals without index*" → Jawaban pasti **Index-Free Adjacency**.
 - Jika pertanyaannya *Red Flags* → Cari AllNodesScan, NodeByLabelScan, dan Eager.

5.3 Langkah Eksekusi Validasi Query via Neo4j Browser / Query Editor

Saat menjumpai soal berbasis dataset hands-on: 1. Buka tab peramban berdampingan (*side-by-side*) antara halaman ujian dan Neo4j Browser (<http://localhost:7474> atau Sandbox). 2. Paste kueri Cypher yang ditanyakan ke dalam kolom kueri neo4j\$. 3. Tekan Ctrl + Enter (atau tombol Play) untuk mengeksekusi kueri. 4. Salin hasil (*output*) persis sesuai format yang diminta ke kolom jawaban ujian.

BAB VI: PENUTUP

6.1 Kesimpulan

1. **Graph Database Neo4j** memberikan solusi arsitektural terhadap masalah performa $O(n)$ pada RDBMS dengan memanfaatkan prinsip *Index-Free Adjacency*, yang memungkinkan penelusuran relasi kompleks berlangsung dalam waktu konstan $O(1)$.
2. **Sertifikasi Neo4j Certified Professional** menguji pemahaman mendalam pada 6 pilar utama: Teori Graph, Sintaks Cypher, Indeks & Constraint, Driver/Keamanan, Query Tuning (Red Flags), dan Eksekusi Agregasi Data.
3. Pemahaman terhadap pola soal serta penerapan strategi manajemen waktu dan pengujian kueri hands-on di Neo4j Sandbox merupakan kunci sukses mencapai nilai kelulusan di atas ambang batas 80% (seperti capaian 97.8% pada pengujian mandiri).

6.2 Saran

1. Peserta ujian disarankan untuk memperbanyak latihan kueri Cypher pada dataset sampel Movie dan Northwind di Neo4j GraphAcademy sebelum mengambil ujian sertifikasi.
 2. Menggunakan dokumen panduan dan kisi-kisi 6 pilar ini sebagai bahan *cheatsheet* atau acuan sinau mandiri bagi mahasiswa maupun praktisi data.
-

Dokumen ini disusun sebagai tugas / nilai tambahan mandiri untuk mata kuliah Data Science.